

Project P10 Research Notebook

Personalized Deep Learning Model for Closed-Loop 40 Hz Entrainment to Optimize
Theta-Gamma Phase-Amplitude Coupling in Alzheimer's Disease

Amaar Chughtai

Valley Christian High School

Synopsys Science and Engineering Fair 2026

Research Period: December 10, 2025 – March 3, 2026

Total Pages: 22

Table of Contents

Section 1: Background Research and Project Selection

Section 2: Dataset Analysis and Technical Implementation

Section 3: Model Architecture Evolution

Section 4: Temporal Prediction Innovation

Section 5: Horizon Sweep Analysis

Section 6: Closed-Loop Controller Integration

Section 7: Advanced Analysis and Validation

Section 8: Statistical Rigor and Reproducibility

Section 9: Conclusions and Future Work

References

Background Framing

By the time this review candidate begins, the project direction was already set: use EEG from 40 Hz auditory entrainment sessions to study whether closed-loop scheduling could improve therapy timing for Alzheimer's disease. Earlier December work included literature review, dataset search, PAC-method reading, and private planning, but those activities are compressed here instead of being presented as official daily notebook entries. The corrected chronology begins at the approval-era fallback anchor used throughout Phase 4.

January 15, 2026: Approval-Era Research Framing and Build Plan

This entry marks the visible start of the corrected notebook. The repository does not contain a tighter approval artifact than the Phase 4 fallback anchor, so I treat January 15 as the fair-facing start date and keep the claims modest.

I used this point to formalize the project around a closed-loop question instead of a broad exploratory neuroscience idea: can EEG be used not just to measure current entrainment, but to help decide when stimulation should be delivered? The immediate planning focus was practical. I needed a pipeline that could load the OpenNeuro data safely, compute θ - γ PAC in a reproducible way, separate subjects across train/validation/test splits, and leave room for a later controller instead of stopping at descriptive analysis.

Repository	basis:	archive/notebooks/LOG_NOTEBOOK.md,
		notebooks/P10_Lab_Notebook_FINAL.md,
		.planning/phases/04-finalize-lab-notebook/04-CONTEXT.md

Working decisions at this stage:

- Keep the notebook anchored to the approval-era timeline rather than the earlier private-prep chronology.
- Treat PAC as the control biomarker, with the understanding that the metric would need careful implementation and auditing.
- Plan the software in clear stages: data loading, preprocessing, PAC computation, model training, and controller logic.

Next step: Start the active build once school workload eases, using gap notes rather than fabricated day-by-day detail for the light-coding period in between.

Gap note (January 16-February 4, 2026): Repository evidence supports continued reading, notebook planning, and school-related delay during this span, but not a dense sequence of day-specific coding milestones. I kept the visible notebook honest by summarizing the period instead of backfilling unsupported entries. The main work during this gap was refining the PAC method choice, planning the pipeline stages on paper, and deciding that the final system would need subject-level leakage protection from the start.

February 6, 2026: First End-to-End Pipeline Build

This was the first strong engineering day in the approval-era record. Three same-day commits show the transition from planning into an actual working pipeline: I built the raw data loader, fixed the ds005048 BIDS path handling, and corrected the MATLAB v7.3 `.set / .fdt` loading path so the repository could finally move from theory to real EEG windows.

The most important technical lesson was low-level and unglamorous: the `.fdt` files had to be reshaped in Fortran order. Until that point, it was possible to load data that looked plausible but was wrong. Once the file handling was corrected, I could trust the downstream steps enough to wire together preprocessing, PAC computation, EEGNet training, the first controller skeleton, personalization logic, simulation, and validation utilities.

Repository **basis:** `git` `log` `entries` `f93c70e,` `9d00715,` `81638ae;`
`archive/notebooks/LOG_NOTEBOOK.md`

What became stable on this day:

- Subject-level splitting as a non-negotiable rule.
- A 7-frontal-channel, 2-second-window representation for the static PAC task.
- A complete raw-data-to-controller software path that could be audited and improved later.

Next step: Train and compare multiple PAC-prediction approaches before assuming that a single architecture choice is enough.

February 16, 2026: Architecture Marathon and Leakage Correction

This was the most instructive day in the project because the biggest gains were not new headline numbers, but better honesty. I tested a broad set of static PAC-prediction approaches in rapid succession: EEGNet variants, handcrafted spectral models, much larger deep models, simple linear baselines, and an early LSTM attempt for temporal prediction. The day initially looked like a breakthrough when a spectral-temporal model appeared to jump to $R^2 = 0.69$, but that result collapsed under inspection because Modulation Index features were leaking PAC information directly into the input.

After removing the circular features, the flattering result disappeared. That was discouraging in the moment, but it established the most durable conclusion of the static phase: the honest single-window ceiling was about $R^2 = 0.287$. Ridge regression matched the stronger deep models, and scaling parameter count upward did not move the ceiling. That changed the interpretation of the problem. The bottleneck was not a missing giant architecture. It was the combination of noisy labels, limited subjects, and the difficulty of inferring PAC from a single short EEG window.

Repository **basis:** `archive/notebooks/LOG_NOTEBOOK.md;`
`notebooks/P10_Lab_Notebook_FINAL.md;`
`docs/submission/reports/PROJECT_ACHIEVEMENT_REPORT.md;` 2026-02-16 commit cluster in git log

Key takeaways:

- Leaked intermediate results were dropped from the notebook as achievements and kept only as debugging lessons.
- $R^2 = 0.287$ became the honest static reference point.

- The project needed a better question than "how do I squeeze more static accuracy out of the same 2-second window?"

Next step: Reframe the project around temporal forecasting, where medium-horizon prediction could matter operationally even if instantaneous estimation remained noisy.

February 17, 2026: Temporal Forecasting Pivot and Multiscale Causal TCN

I treated this day as a genuine project pivot. Instead of trying to predict the current PAC value better, I started asking whether future PAC could be forecast far enough ahead to support proactive control. That shift changed both the dataset design and the model architecture.

The first part of the day was exploratory: longer windows created more temporal continuity than the original 2-second formulation, which made temporal prediction worth revisiting. The second part was the important engineering step. I built a causal temporal pipeline that combined 73 features per timestep: spectral summaries, PAC-history features, and stimulation-context variables extracted from the BIDS event structure. The temporal model itself was a compact MultiscaleCausalTCN with roughly 31,043 parameters, explicit left-only padding, and a receptive field wide enough to cover the 20-second lookback window.

What mattered most was not complexity for its own sake. Every major design choice answered a specific reliability concern: causal padding to block future leakage, multiscale PAC summaries to stabilize short-term noise, and stimulation-state features so the model could reason about whether the brain was in an ON or OFF portion of the protocol.

Repository **basis:** `archive/notebooks/LOG_NOTEBOOK.md;`
`docs/submission/reports/PROJECT_ACHIEVEMENT_REPORT.md;`
`docs/reference/PROJECT_DEEP_DIVE.md;` commits d9213c9, a7299ad, 48c60ad

Why this day was pivotal:

- The problem statement shifted from better instantaneous decoding to usable look-ahead prediction.
- The feature representation became explicitly causal and sequence-based.
- The final project architecture started to look like a controller pipeline rather than a one-off predictor.

Next step: Audit the temporal pipeline carefully before trusting any promising result, especially because target smoothing and PAC-history features could make performance claims easy to overstate.

February 18, 2026: Audit and Methodology Hardening

I used this day to make the project harder to fool myself with. Instead of chasing a new result, I documented assumptions, wrote out methodology choices, and checked the temporal pipeline for the failure modes that had already hurt the static phase: data leakage, split mistakes, and overly flattering interpretations.

This work mattered because the temporal model could easily be oversold. Smoothed targets made some metrics look much better than raw-target forecasting, and PAC-history features had obvious predictive power that needed to be described honestly. Writing the audit notes forced me to separate what the

model truly learned from what was simply easier to predict because of the way the target was defined.

Repository basis: archive/notebooks/LOG_NOTEBOOK.md; commits d1a7609, 9945e3b, 2699c6b, 8ba48d7

What was reinforced:

- Subject boundaries remained intact.
- Temporal targets stayed strictly after the input sequence.
- Train-only normalization and leak-free interpretation had to be part of the story, not just implementation details hidden in code.

Next step: Use the now-audited temporal pipeline to test where the TCN adds real value relative to persistence and linear baselines.

February 19, 2026: Horizon Sweep and Closed-Loop Integration

This was the day the temporal model stopped being merely interesting and started looking useful. I compared persistence, Ridge regression, and the TCN across multiple prediction horizons instead of reporting a single cherry-picked result. That experiment showed a clean pattern: at very short horizons, simple baselines were hard to beat, but at the medium horizons that matter for proactive control, they collapsed while the TCN stayed positive.

The validated horizon summary preserved in the project report is the version I keep here:

Horizon	Persistence R^2	Ridge R^2	TCN R^2
1 s	0.760	0.812	0.735
5 s	-0.267	-0.393	0.254
10 s	-0.256	-0.212	0.278

The practical meaning was clear. If the controller only needed a 1-second look-ahead, sophisticated temporal modeling was not justified. But at 5-10 seconds, the TCN was the only model still carrying usable signal. That made closed-loop integration worth pursuing because it matched the operational timescale where stimulation timing could actually be adjusted proactively.

Repository basis: archive/notebooks/LOG_NOTEBOOK.md; docs/submission/reports/PROJECT_ACHIEVEMENT_REPORT.md; commit 21c54f6

Next step: Build the replay-analysis and robustness layer needed to test the controller on real subject trajectories rather than stopping at model metrics.

February 21, 2026: Replay Framework and Robustness Setup

The corrected notebook keeps this day, but narrows its scope. The repository clearly shows that February 21 was a major validation-framework day: replay-analysis tooling, rigorous-validation scripts, and robustness-oriented documentation all landed here. What it does *not* support is claiming that the final TCN replay result was already locked on this date.

That distinction matters because earlier notebook versions blurred framework construction with the later final controller output. I keep this day focused on the infrastructure that made the final comparison possible: setting up replay analysis, formalizing robustness checks, and preparing the project for a controller-level comparison on real EEG.

Repository **basis:** archive/notebooks/LOG_NOTEBOOK.md;
archive/notebooks/LAB_NOTEBOOK_ERRATA.md; commits 4140775, d65d690, 07acf85, ca3218c, 696b227

Next step: Finish the TCN-integrated replay path and lock the controller metrics on the day the actual validation commits appear, rather than attributing them early.

Gap note (February 22-February 25, 2026): The repository supports continued cleanup, threshold checking, figure generation, and validation plumbing in this span, but it does not justify presenting the final 72.1% controller result before February 26. I treat this as a bridge period between robustness setup and the final replay lock.

February 26, 2026: Real-Data TCN Validation Locked

This is the corrected home for the final controller result. The errata file and the commit history agree that the TCN-integrated replay validation belongs here, not on February 21. By this point the project had moved beyond abstract forecasting metrics and into a direct controller comparison on all 35 subjects' recorded EEG.

The final comparison that I keep in the corrected notebook is the validated controller table preserved in the project report:

Controller	Alignment	Low-PAC Targeting	PAC Gap (x10)
Fixed	45.0%	61.4%	-6.6
Reactive	64.5%	51.7%	+21.1
TCN Predictive	72.1%	82.6%	+30.5
Oracle	100.0%	100.0%	+33.3

Two details were especially important to preserve accurately. First, the fixed controller was not merely weaker; its PAC gap went in the wrong direction, which meant fixed timing was poorly aligned with actual need. Second, the PAC-gap values are dimensionless $\times 10^{\square}$ quantities, not uv^2 . Earlier notebook wording used the wrong units, so I corrected the label instead of carrying it forward.

The controller comparison figure was also mature enough to include here because it belongs to the same final-validation milestone.

![Controller comparison on all 35 subjects](../results/figures/controller_comparison_v2.png)

Figure: Final real-data controller comparison preserved from results/figures/controller_comparison_v2.png. I limited the corrected notebook to this one embedded figure so the review candidate stays sparse and chronology-first.

Repository **basis:** archive/notebooks/LAB_NOTEBOOK_ERRATA.md;
docs/submission/reports/PROJECT_ACHIEVEMENT_REPORT.md;
results/figures/controller_comparison_v2.png; commits 6e575d3, 34c3ac2, beeddf7, 0c8d063,

b14afde, bbe6248, dfb0c16, 1598e7b

Next step: Freeze the science, then translate the validated results into judge-facing documentation without inventing any new claims.

March 1, 2026: Documentation Freeze and Submission Packaging

By early March the main scientific work was finished, so the emphasis shifted to packaging, consistency, and presentation. The corrected notebook keeps this stage concise on purpose. I was no longer changing the scientific story; I was consolidating it into a fair-ready set of documents that judges could read quickly.

The most important packaging choice was narrative discipline. The final materials could now legitimately say that the TCN controller achieved 72.1% alignment, targeted 82.6% of low-PAC windows, and was summarized in the project report as reaching 91% of oracle performance. What I did *not* want to do was turn the notebook back into a polished retrospective paper. The point of this final stage was to preserve the build-debug-validate arc while tightening wording, tables, and figures for external review.

Repository **basis:** notebooks/P10_Lab_Notebook_FINAL.md;
docs/submission/reports/PROJECT_ACHIEVEMENT_REPORT.md

Next step: Keep the corrected source stable, resolve review notes in the sidecars, and only then run the manual PDF export path.

Closing Note

This corrected notebook is intentionally narrower than the preserved legacy version. It does not try to reconstruct every private-prep day, and it does not pretend that every polished conclusion was obvious from the beginning. Instead, it keeps the visible timeline anchored to the approval-era review window, preserves the major engineering and validation milestones in order, and uses the evidence map to constrain what can be claimed with confidence.